Day 7: SOC Threat Simulation and IOC Collection

Simulated File Transfer and Indicator of Compromise Extraction Across an Isolated Lab Network

Field	Details
Intern Name	Amrita Pathak
Date	09 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11:59 PM IST

1. Objective

Perform a harmless, controlled simulation in an isolated lab and collect the observable indicators it produces. Specifically, transfer a test file across a network from one lab machine to another and extract the five categories of evidence a SOC analyst would record for any real incident: username, hostname, timestamp, source IP, and process or file details.

2. Tools and Environment

Item	Detail
Hypervisor	VMware Workstation Pro
File origin (server)	Kali Linux VM (192.168.84.128)
File destination (target)	Metasploitable 2 VM (192.168.84.130)
Network mode	VMware NAT, subnet 192.168.84.0/24
Tools used	ip a, ifconfig, ping, python3 http.server, curl, sha256sum, ls -la, stat

Both machines were placed on the same isolated NAT network used throughout this internship, so no traffic left the host system at any point.

3. Key Terms

- **Indicator of Compromise (IOC)**: a specific, extractable fact tied to an event, such as a username, an IP address, a timestamp, or a file hash, that can be recorded, compared, and shared as evidence. IOCs are the format real incident tickets and threat intelligence are built on, rather than a narrative description of what happened.
- **Source IP**: the network address of the machine that initiated a connection or request. Recording the correct source IP is what allows an analyst to trace an action back to the machine, and ultimately the account, that performed it.
- **Hash (SHA-256)**: a fixed-length string generated from a file's exact contents. Any change to the file, even a single byte, produces a completely different hash. Comparing hashes before and after a transfer is the standard way to prove a file arrived unmodified.
- **HTTP server log**: a record automatically written by a web server for every request it receives, independent of what the requesting machine reports about itself. Because the log is written by the receiving side, it is generally considered more trustworthy evidence than anything self-reported by the requester.
- **stat (Access, Modify, Change)**: three separate timestamps Linux keeps for every file. Access records when the content was last read, Modify records when the content itself last changed, and Change records when the file's metadata, such as permissions or ownership, last changed. These three timestamps do not always move together, and reading them correctly can reveal whether a file was only viewed, actually edited, or had its permissions altered.

4. Why This Task Matters

Earlier tasks in this internship focused on finding and reading events that already existed in a log written by someone else, such as Windows Security log entries in Day 6. This task is different in a specific way. It asks for a simulated event to be generated deliberately, and then for the resulting evidence to be pulled together into the structured format a real SOC ticket actually uses when escalating something for investigation, a short list of concrete, verifiable facts rather than a description of what was observed.

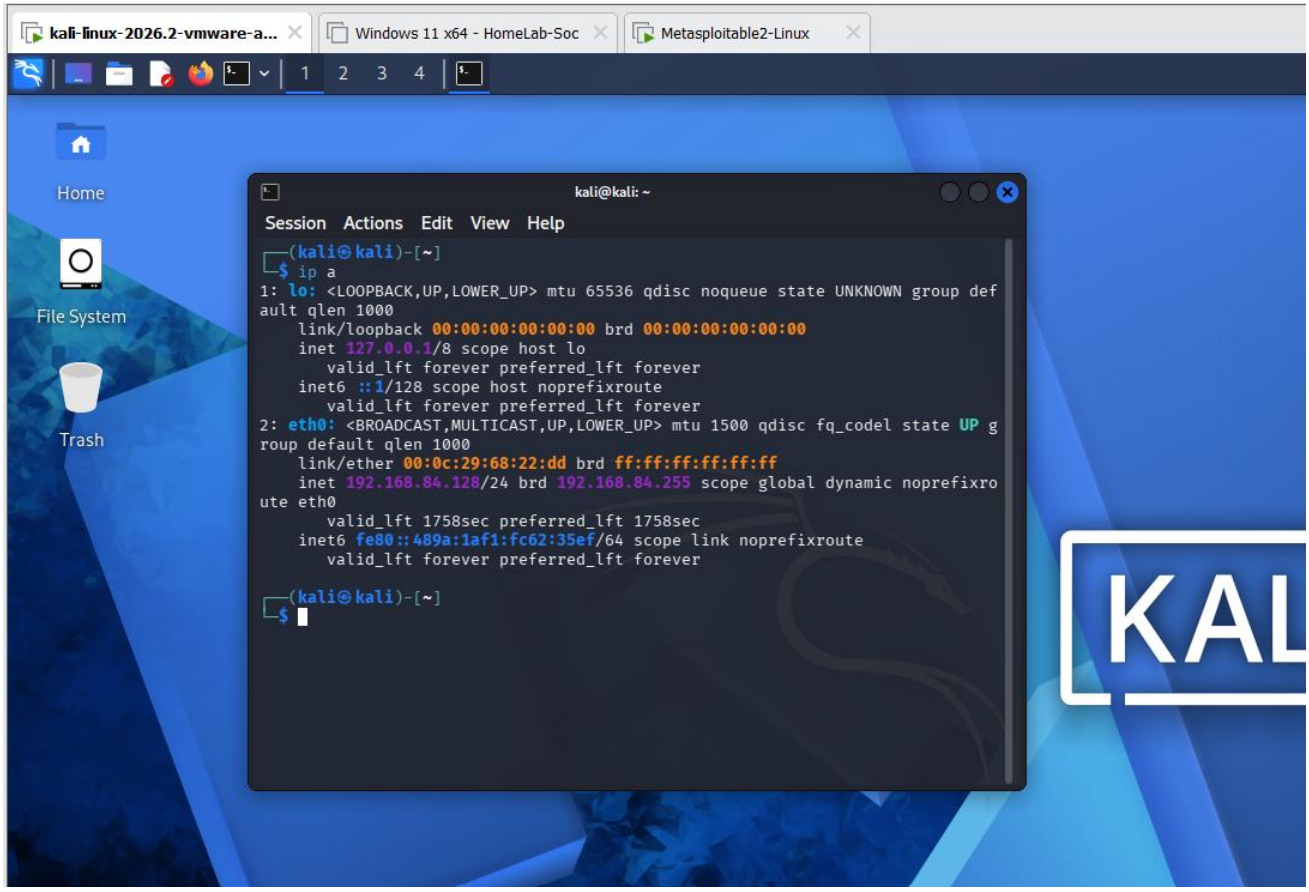
A file transfer was chosen over a repeat of the failed-login scenario from Day 6 for a specific reason. Failed logins had already been generated and analyzed thoroughly on Windows. A file transfer, by contrast, is the only scenario of the two the task describes that naturally produces genuine process and file-level detail, specifically a file hash, a file path, and file size,

none of which had been captured in this internship yet. Simulating the file as something that arrives from another machine over the network, rather than simply typing a command to create it locally, also produces a real, non-loopback source IP, the same kind of address a real analyst would need to trace an actual incident back to its origin.

5. Confirming Network Connectivity

Before any simulation could take place, both machines needed to be identified and confirmed reachable on the same network. Skipping this step is a common mistake under time pressure, since a failed transfer later on could just as easily be a network problem as a real finding, and an analyst who has not already ruled out basic connectivity has no way to tell the difference. On the Kali VM, the following command was run to find its own address.

```
ip a
```



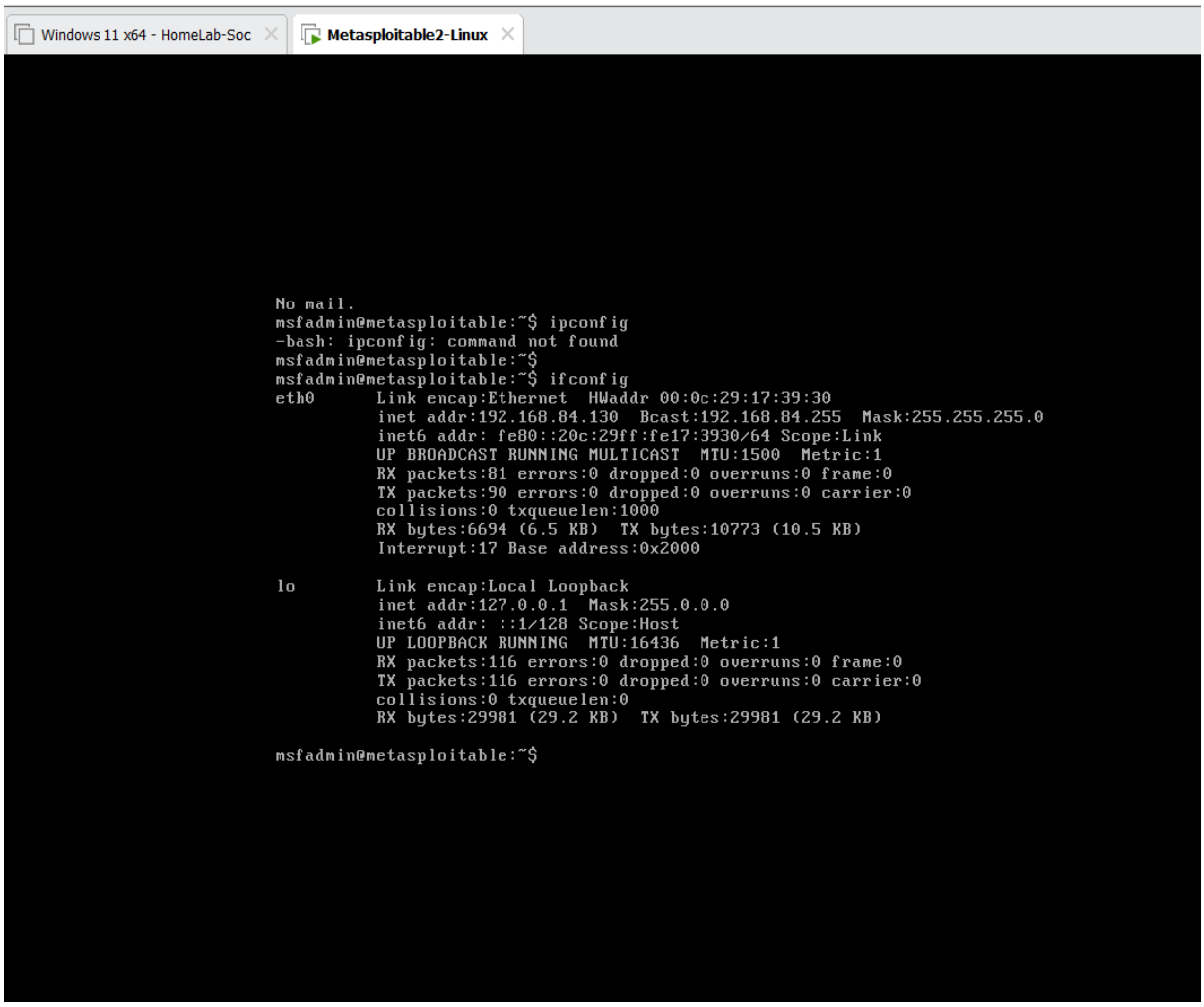
Kali's ip a output, confirming eth0 assigned the address 192.168.84.128.

Reading this output correctly means looking at the right interface and the right line within it, not just the first address on the screen.

- **Interface name: eth0**: the machine's actual network-facing adapter, the one connected to the shared lab network. The output also lists lo, the loopback interface, which only ever talks to itself and was ignored for this purpose.
- **inet 192.168.84.128/24**: the IPv4 address actually assigned to eth0, and the number that matters here. The /24 confirms the subnet size, consistent with the 192.168.84.0/24 network used throughout this lab.
- **scope global**: confirms this address is reachable from other machines on the network, as opposed to a link-local or host-only address that would not be. This is what makes it usable as a genuine source IP later in the simulation.

The same check was performed on Metasploitable, using ifconfig, the traditional Linux utility for this purpose and the one already confirmed to work on this machine after the ipconfig attempt failed.

```
ifconfig
```



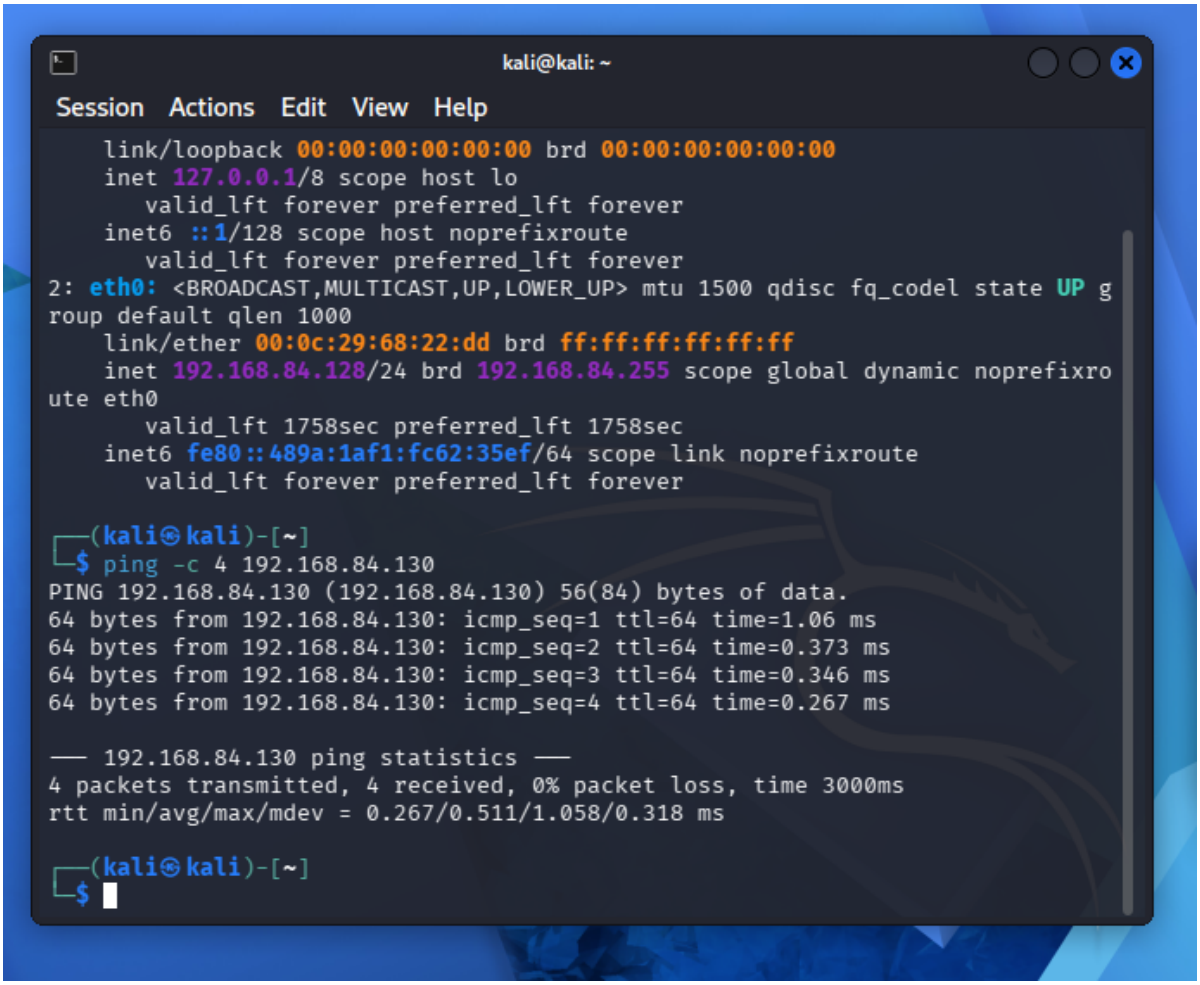
Metasploitable's ifconfig output, confirming eth0 assigned the address 192.168.84.130, in the same subnet as Kali.

- **inet addr:192.168.84.130**: Metasploitable's own address on eth0, in the older ifconfig format rather than the ip command's newer inet syntax. Same information, different tool, which is itself a useful thing to recognize since real environments mix old and new systems constantly.

- **Bcast:192.168.84.255 and Mask:255.255.255.0**: the broadcast address and subnet mask, which together confirm this machine sits in the same 192.168.84.0/24 range as Kali. Two machines on different subnets, even if both technically online, would not be able to reach each other without a router in between.

With both addresses known, connectivity between the two machines was confirmed directly from Kali before proceeding to generate any evidence.

```
ping -c 4 192.168.84.130
```



Four packets sent, four received, 0% packet loss, confirming both machines could reach each other before any simulation began.

- **icmp_seq**: a sequence number identifying each individual ping reply, allowing a lost or out-of-order packet to be spotted immediately rather than just counting totals at the end.
- **ttl=64**: Time To Live, a counter that decreases by one at every network hop a packet crosses. A value of 64 on a reply is typical for a Linux host reached directly on the same local network, with no routers in between decrementing it further.
- **time=1.06 ms (and similar, decreasing values)**: the round-trip time for each reply. Sub-millisecond to low-single-digit millisecond times are what is expected for two virtual machines on the same host talking over a virtual NAT network, rather than across a real physical network.
- **0% packet loss**: the single most important line in this output. Anything above 0% at this stage would mean the network itself is unreliable, and any evidence gathered afterward could not be trusted to reflect the simulation accurately rather than a dropped connection.

6. Preparing and Hashing the Test File

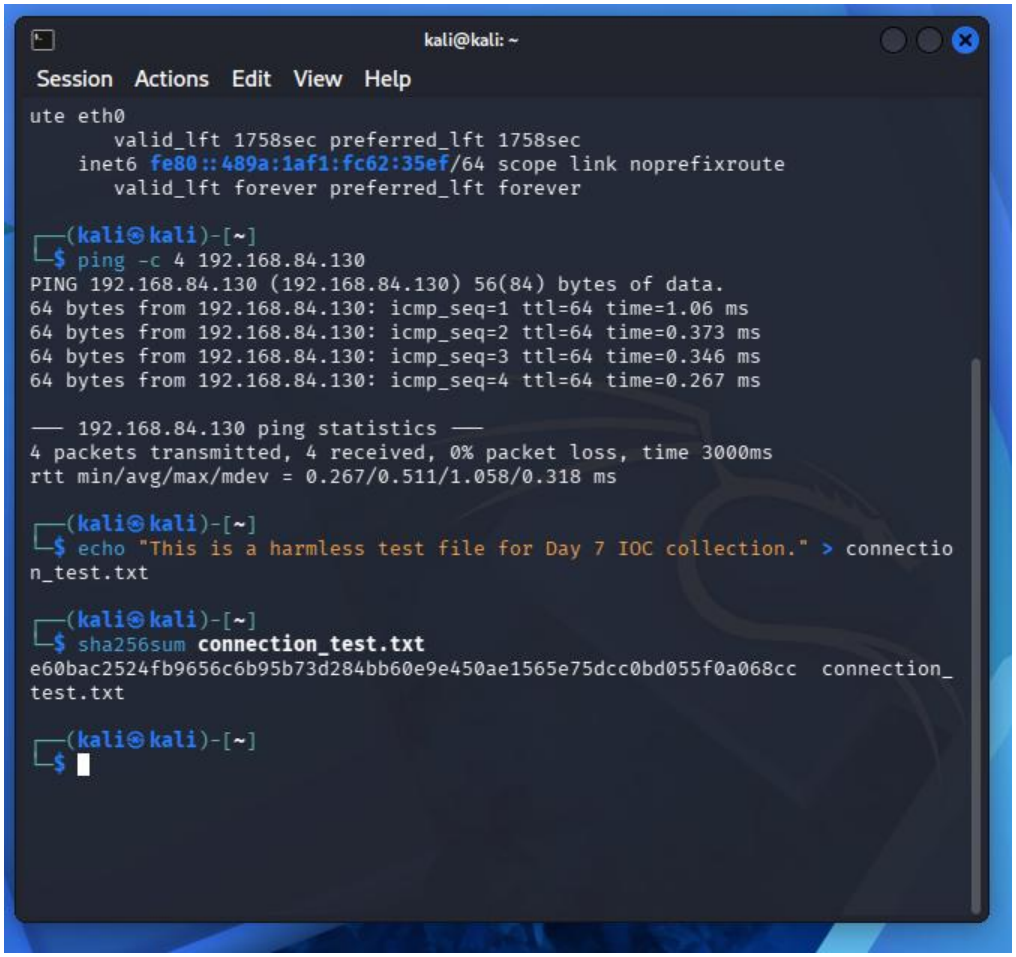
On Kali, a harmless plain-text file was created to serve as the artifact for the simulation. The command below uses echo to generate a short line of text and the > operator to redirect that text into a new file called connection_test.txt, rather than printing it to the screen. The exact wording of the text itself is not important, only that the file has real, non-empty content, since a completely empty file would still hash consistently but would not resemble a realistic dropped artifact.

Immediately after creation, the file's SHA-256 hash was recorded. This ordering matters. Hashing has to happen at the earliest possible point, before the file is touched, moved, or transferred in any way, because the entire value of a hash is as a

snapshot of the file's exact state at one specific moment. A hash taken after the transfer would only prove the file matched itself, not that it survived the trip across the network unchanged.

```
echo "This is a harmless test file for Day 7 IOC collection." > connection_test.txt


sha256sum connection_test.txt
```



The test file created on Kali, with its original SHA-256 hash recorded before transfer: e60bac2524fb9656c6b95b73d284bb60e9e450ae1565e75dcc0bd055f0a068cc.

- **File created silently**: the echo command produces no visible output of its own when redirected with >, since all the text goes into the file rather than the terminal. Seeing the next prompt appear with no error is itself the confirmation the file was written.
- **sha256sum output format**: the hash value is printed first, followed by the filename it belongs to. This pairing matters when hashing multiple files at once, since it is what ties each hash back to the specific file it was generated from.
- **e60bac2524fb9656c6b95b73d284bb60e9e450ae1565e75dcc0bd055f0a068cc**: the resulting hash itself, a 64-character hexadecimal string. This exact value is what gets compared later, in Section 10, against the copy of the file that arrives on Metasploitable. Any difference at all, even a single changed character, would mean the file was altered somewhere in transit.

7. Serving the File Over the Network

Rather than simply copying the file locally, Kali was used to serve the file over HTTP, so that Metasploitable would need to genuinely request and download it across the network. This distinction is the entire reason this scenario was chosen over a simpler local file creation. A file that never leaves one machine has no source IP to record and no independent server log to check against, since the only party who could describe what happened is the same machine being investigated. Serving it over the network turns Kali into an independent witness.

```
python3 -m http.server 8080
```



Kali's HTTP server started and listening on port 8080, ready to serve the test file to any machine on the network that requests it.

- **python3 -m http.server 8080**: starts Python's built-in web server module, using the current directory as its root and port 8080 to listen on. No separate installation was needed, since this module ships with Python itself, which is part of why it is a realistic tool to reach for quickly, by a legitimate administrator or an attacker alike.

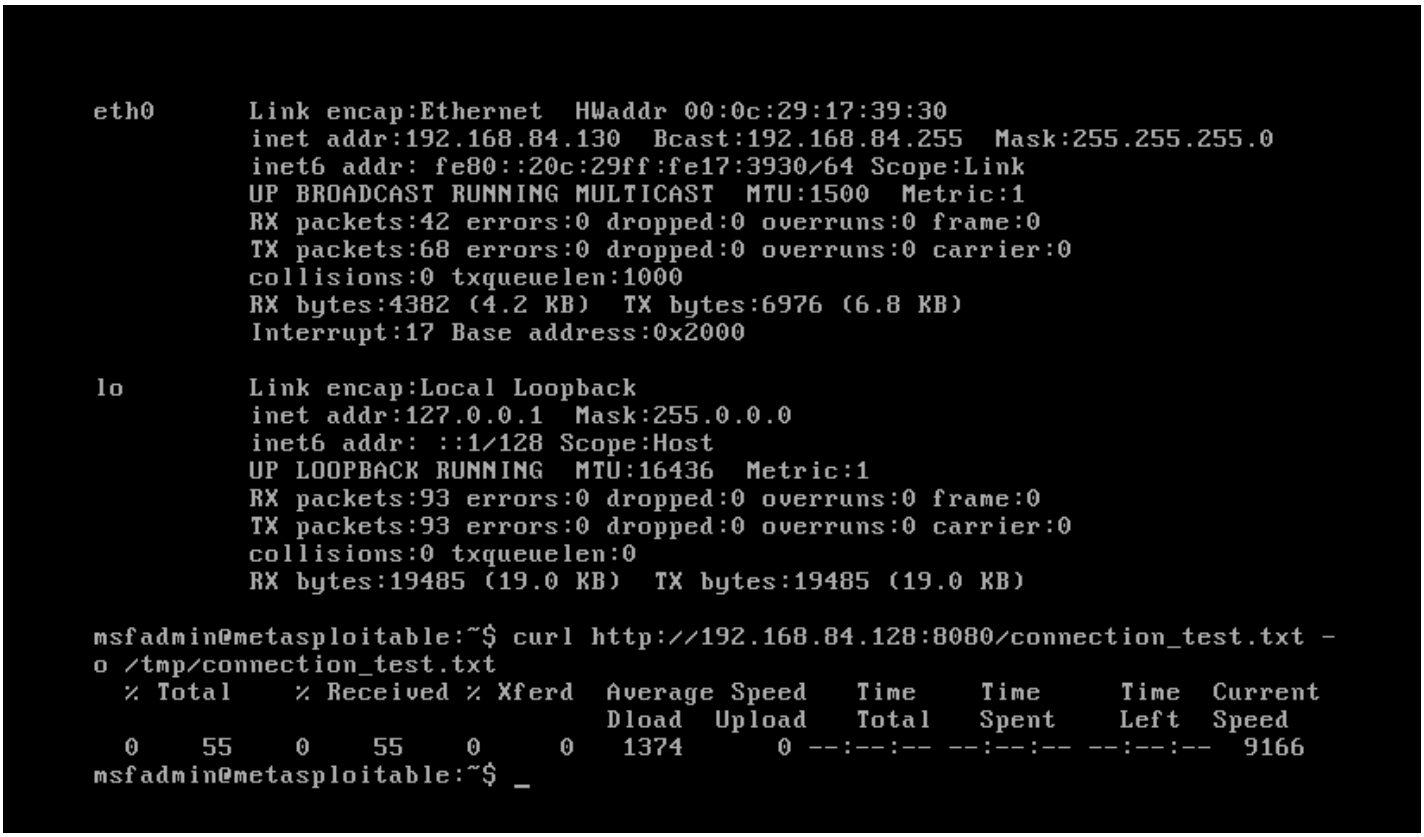
- **Serving HTTP on 0.0.0.0 port 8080**: confirms the server is listening on all of Kali's network interfaces, not just the loopback address. This is what makes the file reachable from Metasploitable at all. A server bound only to 127.0.0.1 would have rejected the connection from another machine entirely.

- **The blinking cursor with no further output**: expected behavior, not a stall. The server process blocks and waits silently until a request arrives, at which point it will print a new log line beneath this one, which is exactly what Section 9 captures.

8. Downloading the File on the Target

From Metasploitable, the file was requested and downloaded from Kali's address. The destination path, /tmp, was chosen deliberately rather than arbitrarily. It is one of the first locations a real analyst checks for dropped or downloaded artifacts, since it is world-writable by default and commonly used, legitimately and otherwise, as a landing spot for files that arrive from outside the system.

```
curl http://192.168.84.128:8080/connection_test.txt -o /tmp/connection_test.txt
```



The curl download completing on Metasploitable, transferring 55 bytes with no errors reported.

- **http://192.168.84.128:8080/connection_test.txt**: the full request target, combining Kali's IP, the port the server is listening on, and the exact filename. Every part of this address had to be correct for the request to succeed, which is itself a small confirmation that the connectivity check in Section 5 was accurate.

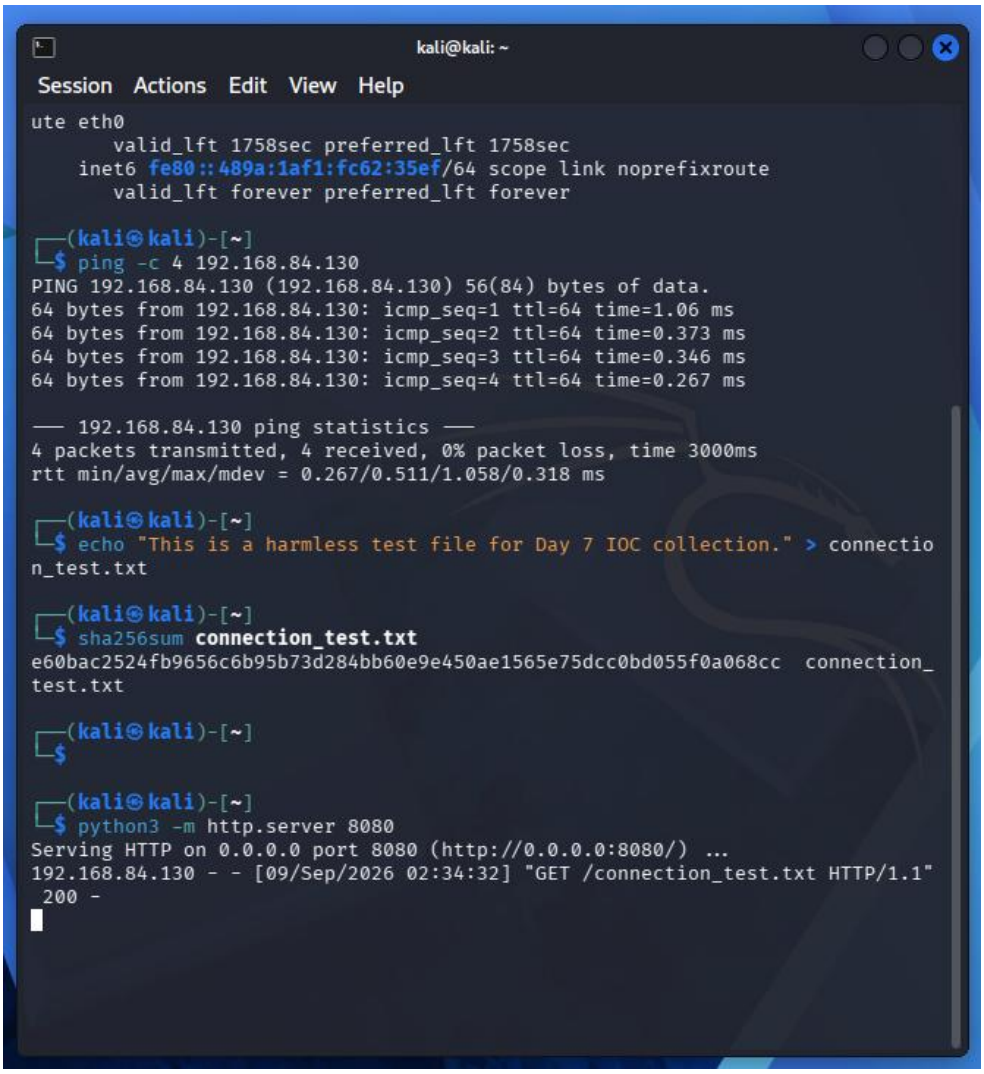
- **-o /tmp/connection_test.txt**: tells curl to save the downloaded content to this specific path instead of printing it to the terminal. Without -o, curl would have dumped the file's text contents directly onto the screen rather than writing a usable file to disk.

- **% Total, % Received, % Xferd, and the speed columns**: curl's built-in progress table. The 100% figures under Total and Received, alongside 55 matching the file's known size, confirm the entire file arrived, not a partial or truncated copy.

- **No error text or non-zero exit reported**: the absence of any curl error message, such as a connection refused or timeout, is itself evidence worth noting, since it confirms the transfer completed cleanly on the first attempt with no retry or interruption.

9. Reading the Server-Side Evidence

The most important single piece of evidence from this entire simulation is the line the Kali HTTP server itself wrote to its own log the moment Metasploitable's request arrived. This line was generated automatically by the server, not typed or reported by the requesting machine, which is exactly what makes it trustworthy evidence rather than a self-reported claim.



The Kali server log entry recording the incoming request from Metasploitable.

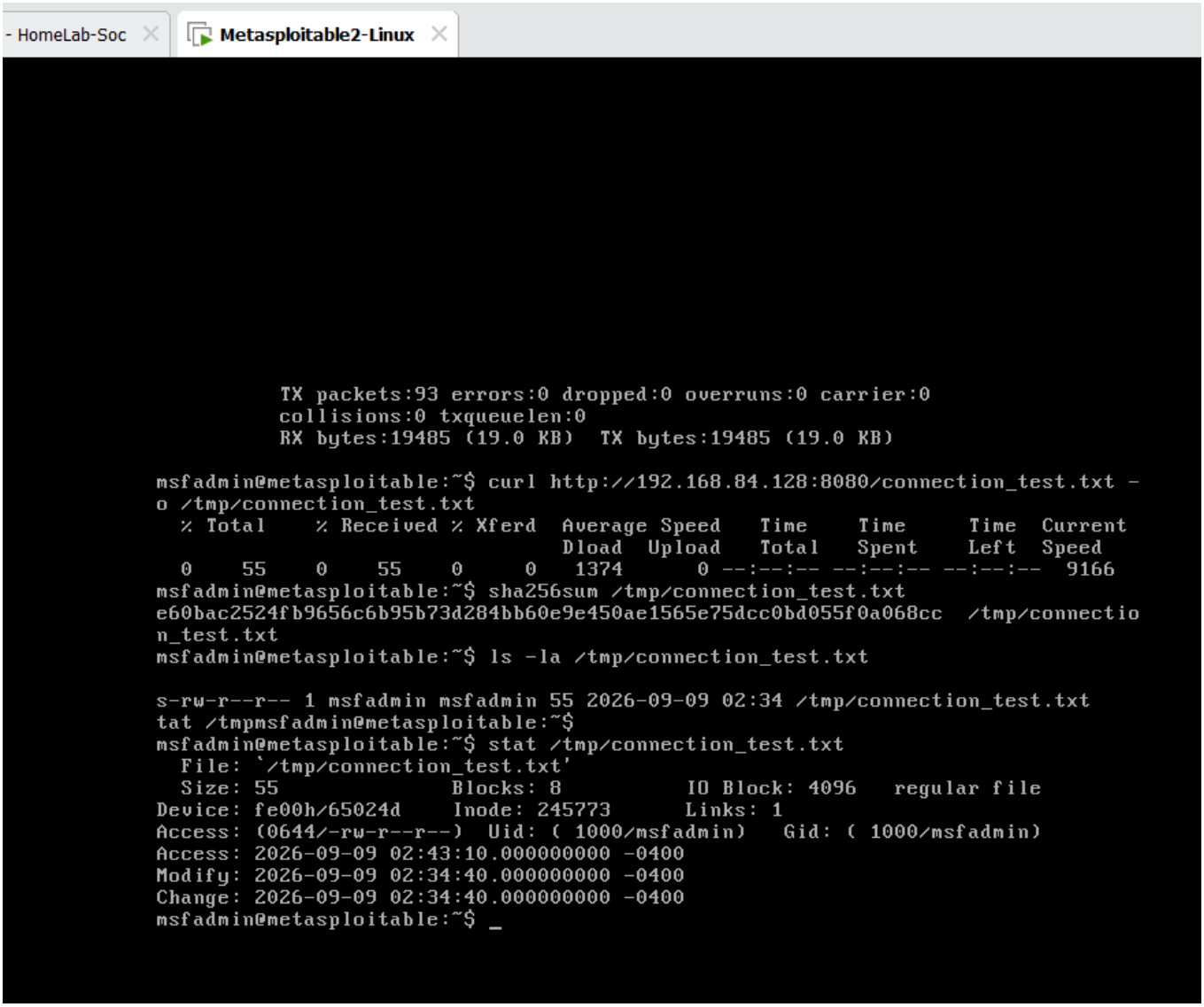
- **192.168.84.130**: the source IP of the request, confirming which machine on the network actually pulled the file.
- **[09/Sep/2026 02:34:32]**: the timestamp, recorded independently by the server at the moment the request was received.
- **"GET /connection_test.txt HTTP/1.1"**: the exact action taken and the specific file requested.
- **200**: the HTTP status code, confirming the request succeeded and the file was served without error.

10. Verifying Integrity and Collecting File Metadata

With the file downloaded, the final step was to confirm it arrived unmodified and to record its full metadata on the target side. The hash was recalculated on Metasploitable and compared against the original value recorded on Kali in Section 6.

```
sha256sum /tmp/connection_test.txt


ls -la /tmp/connection_test.txt
stat /tmp/connection_test.txt
```



Hash verification, file listing, and full stat output on Metasploitable, confirming the file arrived intact and recording its complete metadata.

The recalculated hash on Metasploitable matched the original hash from Kali exactly, **e60bac2524fb9656c6b95b73d284bb60e9e450ae1565e75dcc0bd055f0a068cc** on both sides, proving the file traveled across the network byte for byte unchanged. The remaining fields from ls -la and stat provided the last details needed for a complete evidence picture.

- **Access: 2026-09-09 02:43:10**: the last time the file's contents were read. This is later than the download itself, which is expected, since running sha256sum afterward required reading every byte of the file to compute its hash. The ls -la command run alongside it only reads the file's directory metadata, such as its name and size, and does not touch the file's content, so it does not affect this timestamp on its own.

- **Modify: 2026-09-09 02:34:40**: the last time the file's actual content changed, which corresponds to the moment curl finished writing it to disk. This timestamp is earlier than Access, the correct order, since the content was written first and only read afterward by the verification commands.

- **Change: 2026-09-09 02:34:40**: the last time the file's metadata, such as permissions or ownership, changed. It matches Modify here because both occurred in the same operation, the file being created fresh by curl. A Change timestamp that did not match Modify would instead indicate someone altered the file's permissions or ownership without touching its content, a meaningfully different kind of event.

- **Uid / Gid: 1000/msfadmin**: confirms the file is owned by the account that actually performed the download, consistent with a normal user-initiated curl request rather than, for example, a file dropped by a service running as root.

- **Permissions: -rw-r--r-- (0644)**: standard read and write access for the owner and read-only access for everyone else, the default permission set curl applies to a newly downloaded file.

11. Consolidated IOC Table

Every observable the task asked for, username, hostname, timestamp, source IP, and process or file details, was captured across the two machines. The table below consolidates all of it into the structured format a real SOC ticket would use.

IOC Type	Value	Source
Source IP (requester)	192.168.84.130	Kali HTTP server log
Server IP (file origin)	192.168.84.128	Kali ip a
Destination hostname	metasploitable	Metasploitable shell prompt
Username	msfadmin	Metasploitable file ownership
Timestamp (transfer)	2026-09-09 02:34:32	Kali HTTP server log
Timestamp (file write)	2026-09-09 02:34:40	Metasploitable stat, Modify
Filename / path	/tmp/connection_test.txt	curl output, stat
File size	55 bytes	curl, ls -la, stat
File hash (SHA-256)	e60bac2524fb9656c6b95b73d284bb60e9e450ae1565e75dcc0bd055f0a068cc	Matched on Kali and Metasploitable
Permissions	-rw-r--r-- (0644)	ls -la, stat
Process / tool used	python3 http.server (source), curl (target)	Terminal commands

12. Findings

12.1 Server-side logs are stronger evidence than client-side reporting

The Kali HTTP server log recorded the source IP, timestamp, and requested file automatically, without relying on anything Metasploitable claimed about itself. This is a meaningful distinction in real investigations, since a compromised or malicious client cannot alter what the server itself independently observed and wrote down.

12.2 A matching hash is what actually proves integrity, not just a successful transfer

Curl reporting a completed download with no errors only shows that data arrived. It does not, on its own, prove that data was correct. Only the matching SHA-256 hash on both ends confirmed the file was identical, byte for byte, to the one that left Kali. This is the same logic used in real evidence handling, where a hash mismatch would immediately raise the question of tampering or a corrupted transfer.

12.3 Access, Modify, and Change timestamps tell three different stories

All three timestamps happened to fall close together in this simulation, but they measure genuinely different things. Modify marks when content changed, Change marks when metadata changed, and Access marks when the file was last read. In a real investigation, a Change timestamp that did not match Modify would be a meaningful signal on its own, since it would point to someone altering permissions or ownership without touching the file's actual content.

12.4 A network-based simulation produces IOCs a local-only action cannot

Simply creating the file directly on Metasploitable with a command like touch would have produced file metadata, but no source IP and no server-authored timestamp. Structuring the simulation as a genuine network transfer was what made source IP collection possible at all, which is one of the five observables the task specifically required.

13. Conclusion

A harmless test file was created on Kali, hashed, served over HTTP, and downloaded across the network onto Metasploitable, generating a complete, realistic evidence trail rather than a single isolated event. Every observable the task required was captured directly from command output and log data: the username that owned the file, the hostname of the target machine, the exact timestamps of both the network transfer and the file write, the source IP that initiated the download, and detailed file and process information down to a verified SHA-256 hash.

The exercise also reinforced a broader principle that carries directly into real SOC work. Evidence recorded independently by a server or a system, such as an HTTP access log or a file's own metadata, carries more weight than anything self-reported by the machine being investigated. Cross-checking a hash before and after a transfer, and reading Access, Modify, and Change timestamps as three distinct signals rather than one, are both genuinely transferable skills, not simulation-specific tricks.

Together, these results describe a complete and correctly evidenced simulated file transfer: one source, one target, one artifact, tracked and verified at every stage of its movement across the network, exactly the kind of structured indicator collection a SOC analyst is expected to produce when escalating a real finding for investigation.